T1 - Problemas Clássicos de Concorrência

FPPD — Fundamentos de Processamento Paralelo· Relatório

Integrantes: Guilherme Ryan Domingos Nunes, Lucas Merlini Marchese e Gustavo Henrique da Silva
Github: https://github.com/ryandnunes00/TRABALHO1_FPPD.git

1. Visão geral e metodologia

Os dois problemas foram implementados em go usando biblioteca padrão (goroutines, channels, select, sync.WaitGroup, sync/atomic), seguindo o estilo dos exemplos do professor. Os programas recebem os parâmetros por linha de comando (-n, -r nos filósofos; -p, -c, -k no produtor/consumidor). todos os programas executados com go run -race; para cada problema

foram feitas 5 execuções independentes (filósofos: N=5, R=20; produtor/consumidor: P=3, C=3, 200 itens por produtor, K E {1, 10, 100}). Tabelas mostram médias das 5 execuções. Como são poucas execuções e o tempo é simulado com time.Sleep, os valores dependem da máquina e as conclusões são baseadas na interpretação dos resultados observados.

2. Problema 1 — Jantar dos Filósofos

2.1 Solução e decisões de projeto

Cada garfo é um channel chan struct{} com buffer 1 contendo um token: pegar o garfo é receber do channel e devolver é enviar o token de volta. O filósofo i usa os garfos i (esquerdo) e (i+1) mod N (direito). Entre pegar o primeiro e o segundo garfo há uma pausa de 2 ms (30 ms na versão base): ela não altera a lógica, apenas aumenta a chance de o escalonador intercalar os filósofos no pior caso, tornando o deadlock da versão base reproduzível (o que ocorreu nas 5 execuções). Foram implementadas quatro versões:

Versão	Funcionamento	Condição de Coffman
base	todos pegam esquerda → direita	nenhuma (as 4 valem) → deadlock
hierarquia	hierarquia de recursos [3]: sempre pega primeiro o garfo de menor número (na prática só o filósofo N−1 inverte a ordem)	espera circular
limite	um channel com N−1 tokens atua como “porteiro”: no máximo N−1 filósofos disputam garfos; com N−1 à mesa sempre sobra um garfo livre	espera circular
garcom	goroutine central recebe pedidos e só autoriza quando os dois garfos estão livres; pedidos em fila FIFO, e um pedido pendente “reserva” seus garfos, impedindo que pedidos mais novos o ultrapassem	posse e espera (hold and wait)

Exclusão mútua e não-preempção são inerentes ao problema (um garfo só está com um filósofo e não pode ser tomado à força); por isso as estratégias atacam a posse e espera ou a espera circular. O garçom é o único dono da fila e do vetor de garfos ocupados, logo dispensa mutex. Cada goroutine escreve apenas nas suas posições dos vetores de resultados, sem data race.

2.2 Análise de deadlock e starvation

Versão	Deadlock	Starvation	Justificativa
base	Ocorre nas 5	não se aplica (ninguém progride)	todos seguram 1 garfo e esperam o outro: as 4 condições de Coffman valem simultaneamente. O diagnóstico impresso mostrou os 5 filósofos em "segura 1 garfo, espera o outro".
hierarquia	Não ocorre	não observada	espera circular não fecha: o filósofo 4 disputa o garfo 0 antes do garfo 4. A garantia contra starvation depende de os channels do runtime Go atenderem os receptores em ordem FIFO (detalhe de implementação, não da especificação).
limite	Não ocorre	não observada	com ≤ N−1 à mesa ao menos um filósofo obtém os dois garfos. Mesma ressalva sobre a ordem de atendimento do channel "porteiro".
garcom	Não ocorre	Não ocorre	atribuição atômica dos dois garfos e prioridade ao pedido mais antigo: um pedido só espera por pedidos anteriores que o conflitam.

Observação empírica: nas 15 execuções das três estratégias sem deadlock, todos os filósofos concluíram as R=20 refeições e a espera máxima observada ficou em torno de 3 vezes a espera média (máximo geral de 31,6 ms). Execuções finitas não provam ausência de starvation; a garantia só vale por projeto no garçom.

2.3 Fairness: dados de execução

Estratégia	Deadlock	Refeições por filósofo	Espera média (ms)	Espera máx. (ms)	Jain (média ± dp)	Duração média (ms)
base	5/5	0 (travou)	—	—	n/a	≈1100–1200 (até detectar)
hierarquia	0/5	20 (todos)	5,9	19,0	0,9175 ± 0,0199	280
limite	0/5	20 (todos)	8,8	31,6	0,9982 ± 0,0012	304
garcom	0/5	20 (todos)	11,4 (8,5*)	31,4 (19,8*)	0,9973 ± 0,0022	356 (297*)

Estratégia	F0	F1	F2	F3	F4	Máx/mín
hierarquia	7,90	5,56	4,37	3,80	8,02	2,11
limite	9,05	9,00	8,75	8,63	8,69	1,05
garcom*	8,31	8,39	8,63	8,28	8,79	1,06

Como todos os filósofos fazem exatamente R refeições, a fairness aparece na distribuição das esperas, medida pelo índice de Jain [5], $J = (\Sigma x)^2 / (n \cdot \Sigma x^2)$, que vale 1 quando todos esperam igual. limite ($J \approx 0{,}9982$) e garcom ($J \approx 0{,}9973$) são praticamente equânimes. hierarquia é a menos justa ($J \approx 0{,}9175$, e em nenhuma das 5 execuções passou de 0,95): os filósofos 0 e 4 esperaram em média 7,9 e 8,0 ms, contra 3,8 ms do filósofo 3, ou seja, mais que o dobro. A assimetria da hierarquia concentra a disputa no garfo 0, que é o primeiro recurso pedido tanto pelo filósofo 0 quanto pelo 4 (nossa explicação para o padrão, não verificada por instrumentação adicional). Conclusão: evitar deadlock não implica fairness.

Por outro lado, a hierarquia teve a menor espera média (5,9 ms contra 8,8 do limite e 8,5 do garçom, este sem o outlier). Nossa hipótese é que, com a hierarquia, um filósofo pode já segurar o primeiro garfo enquanto espera o segundo, sobrepondo espera e posse, enquanto limite e garçom são mais conservadores e acrescentam custo de coordenação (fila do porteiro, ida e volta ao garçom). Na execução 2 o garçom apresentou espera média de 22,8 ms e duração de 592 ms, cerca de 2,7× as demais; como algoritmo e parâmetros são os mesmos, atribuímos a interferência do sistema operacional (não investigamos a causa). Nessa execução todos os filósofos foram atrasados por igual (Jain = 0,9996), então a fairness não foi afetada.

2.4 Encerramento ordenado

Cada filósofo executa exatamente R refeições e retorna (defer wg.Done()); a main espera no WaitGroup. O garçom é encerrado depois: close(fim) seguido de <-terminou, garantindo que a goroutine finalizou. Na versão base, que trava de propósito, um watchdog observa um contador atômico de refeições; se ele não muda por 1 s, declara deadlock, imprime o estado de cada filósofo e fecha o channel abort. Toda espera por garfo é um select entre o garfo e abort, então os filósofos bloqueados acordam, devolvem o garfo que seguravam e retornam: mesmo o deadlock termina sem goroutines bloqueadas.

3. Problema 2 — Produtor/Consumidor com buffer limitado

3.1 Solução e decisões de projeto

As duas versões implementam a mesma interface Buffer (Put, Take, TakeTimeout, Close, Len), de modo que produtores, consumidores, medições e verificação são idênticos e a comparação é justa. Versão com channel: o buffer é make(chan int, K); o bloqueio com buffer cheio/vazio é o do próprio Go. Versão com semáforos (algoritmo clássico, Ben-Ari [1]): array circular de tamanho K e quatro semáforos: notFull (inicia em K, conta vagas), notEmpty (inicia em 0, conta itens), mutexIn e mutexOut (binários, protegem as posições de escrita e leitura). Cada semáforo é um channel de tokens (Wait = receber, Signal = enviar), o que evita dependência externa e permite usar select sobre notEmpty.

O produtor faz notFull.Wait → mutexIn.Wait → escreve e avança in → mutexIn.Signal → notEmpty.Signal; o consumidor, o simétrico com notEmpty, mutexOut e notFull. Por que contar não basta: sem os mutexes, dois produtores com vagas reservadas leriam o mesmo in e escreveriam na mesma posição, e dois consumidores retirariam o mesmo item. Por que dois mutexes: um produtor e um consumidor mexem em índices e posições

distintas e podem trabalhar simultaneamente; os semáforos de contagem ordenam os acessos (uma posição só é lida após notEmpty.Signal e só é reescrita após notFull.Signal), o que o detector de race confirmou. Consumo com timeout: o consumidor 0 faz select entre receber um item e time.After(15 ms); no timeout executa o comportamento alternativo (registra e tenta de novo). Os produtores iniciam 40 ms atrasados, o que provoca ≈2 timeouts por execução (coluna da Tabela 3). Os tempos de produção e consumo seguem distribuição exponencial (média 2 ms) para gerar rajadas irregulares, cenário em que a capacidade K faz diferença.

3.2 Deadlock e starvation

Deadlock: não ocorre em nenhuma versão. Na versão com semáforos, a vaga/item é sempre reservada (notFull/notEmpty) antes de adquirir o mutex, então nenhuma goroutine segura um mutex enquanto espera por vaga ou item (se a ordem fosse inversa, um produtor poderia reter o mutex esperando uma vaga). Os mutexes só protegem trechos curtos que não bloqueiam. Starvation: não observada. Em cada versão (5 execuções × 3 valores de K × 3 consumidores = 45 contagens) os consumidores retiraram quantidades próximas, entre 181 e 218 dos 600 itens (média ideal: 200), e todos os produtores concluíram. Assim como no Problema 1, essa evidência é empírica e

depende de o runtime atender os bloqueados em ordem FIFO.

3.3 Efeito da capacidade K do buffer

Versão	K	Vazão (itens/s) média ± dp	Tempo (ms)	Ocupação média (itens)	Ocupação (% de K)	Itens por consumidor (mín–máx, 15 valores)	Timeouts (méd./exec.)
channel	1	1030 ± 12	583	0,44	44,1	187–211	2,0
channel	10	1112 ± 36	540	3,28	32,8	182–218	2,0
channel	100	1135 ± 22	529	8,25	8,2	185–216	2,0
semaforo	1	1015 ± 15	592	0,44	44,0	185–209	2,0
semaforo	10	1103 ± 48	545	4,27	42,7	190–216	2,2
semaforo	100	1093 ± 28	549	8,98	9,0	181–214	2,0

K=1 → K=10: a vazão sobe 8,0% no channel (1030 → 1112 itens/s) e 8,7% no semáforo (1015 → 1103), variação maior que o desvio-padrão. Com K=1 produtor e consumidor ficam quase acoplados (um rendezvous por item): qualquer rajada de um lado bloqueia o outro, e o buffer passa ~44% do tempo cheio. K=10 → K=100: a variação é pequena (2,0% no channel e -0,9% no semáforo, dentro do desvio-padrão): há saturação. Com K=100 a ocupação média absoluta fica em ≈8–9 itens (≈8–9% de K): como as taxas médias de produção e consumo são equilibradas, o buffer só acumula rajadas de poucas dezenas de itens e a capacidade extra fica ociosa. O desacoplamento, portanto, cresce com K, mas com retorno decrescente: um buffer pequeno (≈10) já absorve a maior parte da variabilidade; mais capacidade só ajudaria com períodos longos em que um lado fosse mais rápido que o outro. A vazão (≈1100 itens/s) fica abaixo do teto de ≈1500 itens/s (3 produtores, 2 ms por item), provavelmente por causa do custo de time.Sleep e do -race.

Channel × semáforos: o desempenho é equivalente. Em K=1 e K=10 as diferenças (1,5% e 0,8%) estão dentro do desvio-padrão; em K=100 o channel foi 3,8% mais rápido, mas com n=5 não é conclusivo. O tempo é dominado pelo trabalho simulado, não pela sincronização. A diferença prática é de simplicidade e segurança: o channel resolve exclusão mútua e contagem em uma linha, enquanto a versão com semáforos exige quatro semáforos e a ordem correta das operações (erros produzem posições sobrescritas ou itens duplicados). As ocupações médias

dos dois diferem um pouco em K=10 (3,3 × 4,3), mas Len() não mede exatamente a mesma coisa (no semáforo, são as permissões de notEmpty) e não interpretamos essa diferença.

3.4 Encerramento ordenado e verificação

A main espera todos os produtores (wgProd.Wait()) e só então chama Close(). Channel: close(ch); os consumidores continuam recebendo até drenar o buffer e só então veem "fechado", sem perder itens. Semáforos: não existe close; a main insere C "pílulas de veneno" (valor −1) pelo Put normal, depois dos produtores. Como o buffer é FIFO, cada pílula só é retirada após todos os itens reais, e o consumidor que a lê termina. Em seguida a main espera os consumidores (wgCons.Wait()). Cada item tem um id único e um vetor vistos[id] (incrementado com atomic) é conferido ao final: total produzido = total consumido e cada item foi consumido exatamente uma vez; qualquer divergência imprime ERRO e encerra com código 1. As 30 configurações medidas (2 versões × 3 valores de K × 5 execuções) passaram na verificação.

4. Uso de ferramentas de IA

Ferramenta utilizada: Claude (Anthropic)

1) Orientou a resolver erros de instalação do Go e do gcc no Windows para habilitar o -race cujo o problema foi pasta duplicada de go no c://
2) Criou comando no terminal que gera os txt
3) Melhorou codigo que possuia gargalos e feiuras

5. Referências

Exemplos fornecidos pelo professor e claude para concerta alguns erros de lógica no switch que tenho dificuldade.

Deixei em anexo no github os conteudos utilzados.